

Scala Coding Challenge Test (2 Hours)

Total Marks: 100

Duration: 120 Minutes

Section	Type	Questions	Marks
Section A	Coding Fundamentals	3	30
Section B	Collections & Functional Programming	3	30
Section C	OOP & Design	2	20
Section D	Debugging	2	20

Section A – Coding Fundamentals (30 Marks)

Question 1 (10 Marks)

Write a Scala program that:

1. Accepts a number **N**
2. Prints all numbers from **1 to N**
3. Prints:
 - Sum of numbers
 - Average

Example

Input

N = 5

Output

Numbers: 1 2 3 4 5

Sum = 15

Average = 3

ANSWER:

```
scala> import scala.io.StdIn.readInt
import scala.io.StdIn.readInt
```

```
scala> println("enter the number")
enter the number
```

```
scala> println("enter the number");val num:Int=readInt()
enter the number
```

```
val num: Int = 5
```

```
scala> var sum:Int=0
```

```
var sum: Int = 0
```

```
scala> for(i<-1 to num){
  | sum=sum+i
  | println(i)
  | }
```

```
1
```

```
2
```

```
3
```

```
4
```

```
5
```

```
scala> println(sum)
15
```

```
scala> println(sum/num)
3
```

```
\
```

```
scala> def sum()={
  | println("enter the number")
  | val num:Int =readInt()
  | var sum:Int=0
  | for (i<-1 to num){
  | sum=sum+i
  | print(i)
  | }
  | println()
  | println("sum="+sum)
  | println("avg="+sum/num)}}
scala> sum()
```

```
enter the number
```

```
12345
```

```
sum=15
```

```
avg=3
```

```
scala> sum()
enter the number
12345678910
```

```
sum=55  
avg=5
```

Suggestion : you can write it in a same code instead of making it long

Question 2 (10 Marks)

Write a program to check whether a **number is a palindrome**.

Example

Input: 121

Output: Palindrome

ANSWER:

```
scala> def pali(num:Int)={  
  | var dup:Int=num  
  | var rev:Int=0  
  | while(dup>0){  
  | var temp:Int=0  
  | temp=dup%10  
  | rev=rev*10+temp  
  | dup=dup/10}  
  | if (num==rev){  
  | println("palindrome")}  
  | else{  
  | println("no palindrome")}  
  | }  
def pali(num: Int): Unit
```

```
scala> pali(12)  
no palindrome
```

```
scala> pali(121)  
palindrome
```

Suggestion : Its good , working well

Question 3 (10 Marks)

Write a program to generate the **Fibonacci series up to N terms**.

Example

Input: 7

Output:

0 1 1 2 3 5 8

ANSWER:

```
scala> def fib(num:Int)={  
  | var a:Int=0  
  | var b:Int=1  
  | var n:Int=num  
  |  
  | if (num==0){  
  | println(a)}  
  |  
  | else if (num==1){  
  | println(b)}  
  | else{  
  | while (n>0){  
  | print(a+" ")  
  | var temp:Int=a  
  | a=b  
  | b=temp+b  
  | n=n-1}}}  
  | print(a+" ")  
  |  
  ^
```

On line 13: warning: method + in class Int is deprecated (since 2.13.0): Adding a number and a String is deprecated. Use the string interpolation `s"\$num\$str"`

def fib(num: Int): Unit

```
scala> fib(7)
```

0 1 1 2 3 5 8

Suggestion : you can make code more concise using recursion

Section B – Collections & Functional Programming (30 Marks)

Question 4 (10 Marks)

Given a list:

```
val numbers = List(10,20,30,40,50)
```

Write Scala code to:

1. Multiply each element by 2

2. Filter numbers **greater than 50**
3. Print the result

Expected Output

List(60,80,100)

ANSWER:

```
scala> def filter(l:List[Int]): Unit= {  
  | var result:List[Int]=l.map(_*2).filter(_>50)  
  | println(result)}  
def filter(l: List[Int]): Unit
```

```
scala> filter(List(10,20,30,40,50))  
List(60, 80, 100)
```

```
scala> filter(List(15,25,35,45,55))  
List(70, 90, 110)
```

Suggestion : yeah it's perfect

Question 5 (10 Marks)

Given a sentence:

"scala is powerful and scala is functional"

Write a Scala program to **count occurrences of each word**.

Expected Output

```
scala -> 2  
is -> 2  
powerful -> 1  
and -> 1  
functional -> 1
```

ANSWER:

```
scala> def wordCount(text: String): Unit = {  
  |  
  | val words = text.split(" ")  
  |  
  | for (word <- words.distinct){  
  | val count=words.count(_==word)  
  | println(word+"<-" +count)  
  | }  
  | }
```

```
def wordCount(text: String): Unit
```

```
scala> wordCount("scala is functional and scala is powerful")
scala<-2
is<-2
functional<-1
and<-1
powerful<-1
```

Suggestion : the code is optimized and well

Question 6 (10 Marks)

Write a Scala program to **remove duplicates from a list and sort it**.

Example

Input:

List(4,2,1,3,2,4,5)

Output:

List(1,2,3,4,5)

ANSWER:

```
scala> def re(lst:List[Int])={
  | println(lst.distinct.sorted)}
def re(lst: List[Int]): Unit
```

```
scala> re(List(1,2,3,2,4,4,5,1,9))
List(1, 2, 3, 4, 5, 9)
```

Suggestion : Perfect

Section C – OOP Concepts (20 Marks)

Question 7 (10 Marks)

Create a **Student class** with:

Attributes:

- id
- name
- marks

Functions:

- display student details
- check if student **passed (marks > 40)**

Example Output

Student: Rahul

Marks: 55

Status: Pass

ANSWERS:

```
scala> class student(name:String,id:Int,mark:Int){  
  | def pass()={  
  | if (mark>40){  
  | println("passed")}  
  | else{println("not passed")}}  
  | }
```

```
class student
```

```
scala> val n1=new student("anvi",1,45)  
val n1: student = student@5ff7631f
```

```
scala> n1.pass()  
passed
```

```
scala> val n2= new student("aby",2,35)  
val n2: student = student@5a33ba9b
```

```
scala> n2.pass()  
not passed
```

Question 8 (10 Marks)

Create a **BankAccount** class.

Functions:

- deposit(amount)
- withdraw(amount)
- checkBalance()

Example

Deposit 1000

Withdraw 200

Balance: 800

ANSWER:

```
scala> class bank_account(){  
  | var balance=0  
  | def deposit(num:Int)={  
  | balance=balance+num  
  | println("deposited"+num+",current balance =" +balance)  
  | }  
  | def withdraw(num:Int)={  
  | balance=balance-num  
  | println("withdrawn " +num,"current balance =" +balance)}  
  | def show_balance()={  
  | println("current balance"+balance)}  
  | }  
class bank_account
```

```
scala> val acc1=new bank_account  
val acc1: bank_account = bank_account@51d84d7f
```

```
scala> acc1.show_balance()  
current balance0
```

```
scala> acc1.deposit(100)  
deposited100,current balance =100
```

```
scala> acc1.withdraw(50)  
(withdrawn 50,current balance =50)
```

```
scala> acc1.show_balance()  
current balance50
```

Suggestion : The code is incomplete , add the withdraw function properly

Section D – Debugging (20 Marks)

Question 9 (10 Marks)

Fix the following code.

```
val numbers = List(1,2,3,4)
```

```
val result = numbers.filter(x => x = 2)
```

```
println(result)
```

Expected Output

List(2)

ANSWER:

```
val numbers = List(1,2,3,4)
```

```
val result = numbers.filter(x => x == 2)
```

```
println(result)
```

```
#no comments
```

Question 10 (10 Marks)

Identify and fix errors.

```
class Person(name:String){  
  println("Person name is "+name)  
}
```

```
val p = new Person
```

#ANSWER:

```
val p = new Person("anvin")
```

There should be argument

```
# no comments
```

Bonus Challenge (Optional – 10 Marks)

Create a **Mini Student Management System**

Features:

- Add student
- Display students
- Search student by name

Use:

- Case class
- List collection
- Functions

Criteria	Marks
Code correctness	40
Logic implementation	30
Code readability	10
Use of Scala features	10
Efficiency	10